포트폴리오 — 웨이퍼 측정 시퀀스 시뮬레이터 (박천일)

Oct 13, 2026 · @박천일

1. 한 눈에

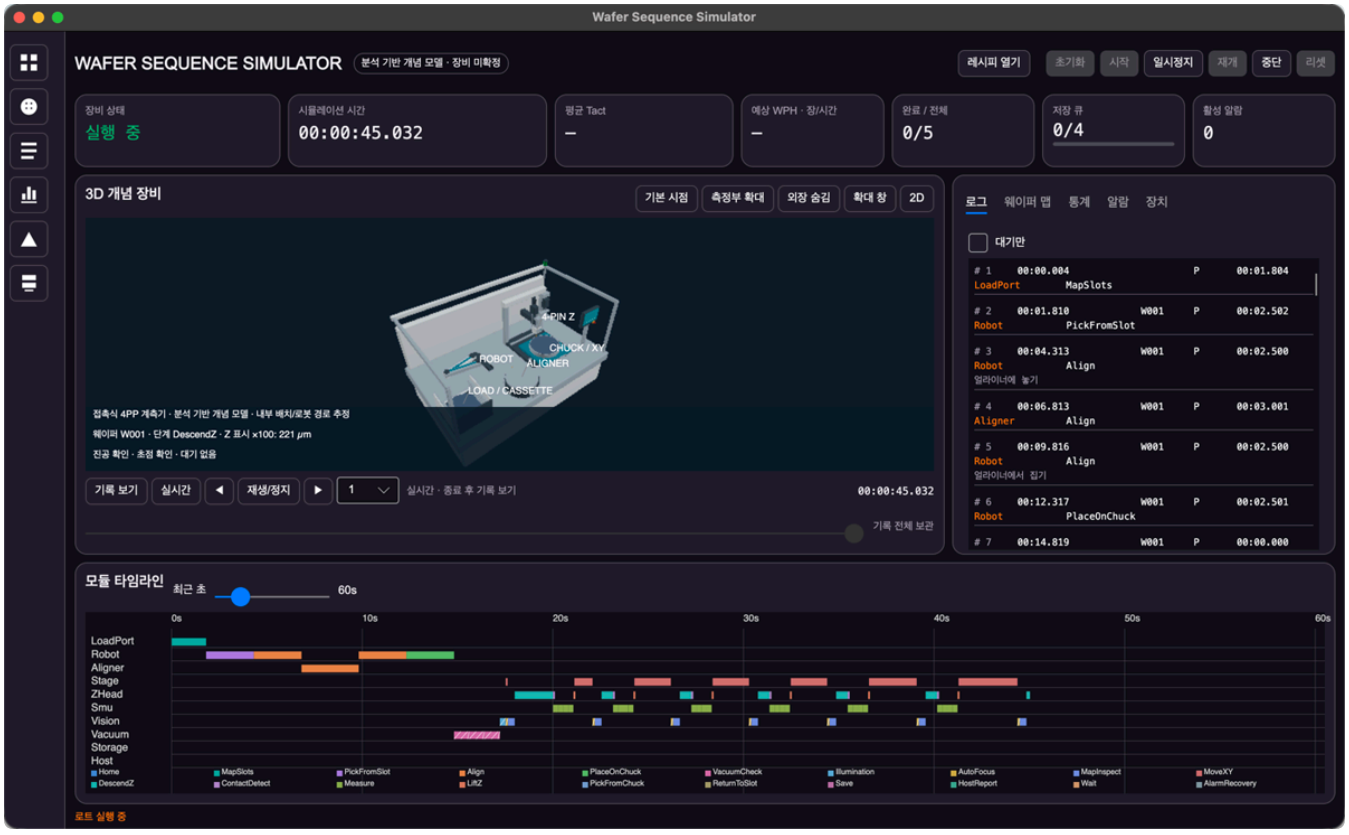
접촉식 4점 프로브 면저항 계측 장비의 **제어 시퀀스·처리량(Tact/UPH)·측정 데이터**를 가상 장치 위에서 재현하는 C#/.NET 10 데스크톱 시뮬레이터다. 엔진 하나를 WinForms(장비 HMI 표준 스택)와 Avalonia(맥 시연용) 두 화면이 공유하고, 장비는 3D 개념 모델로 돌려 보며 실행 기록을 타임슬라이더로 되감을 수 있다. 모든 수치는 가상이며 특정 회사·장비의 값이 아니다.

항목	내용
저장소	https://github.com/Parkchenil/wafer-sequence-sim (공개, MIT 아님 — 라이선스 미지정)
기술	C# 13 / .NET 10, WinForms(GDI+), Avalonia 12, xUnit, SQLite, GitHub Actions(ubuntu + windows)
규모	프로젝트 8개, 테스트 89건, 3D 모델 삼각형 22,608개(191메시)
제작	2026-10-12 저녁 3시간에 1차 완성, 이어 3D·기록 재생 고도화
보는 법	README의 시연 gif, CI 아티팩트(윈도우 러너가 진짜 엔진으로 찍은 WinForms 화면), 또는 `dotnet run --project src/App.Avalonia`

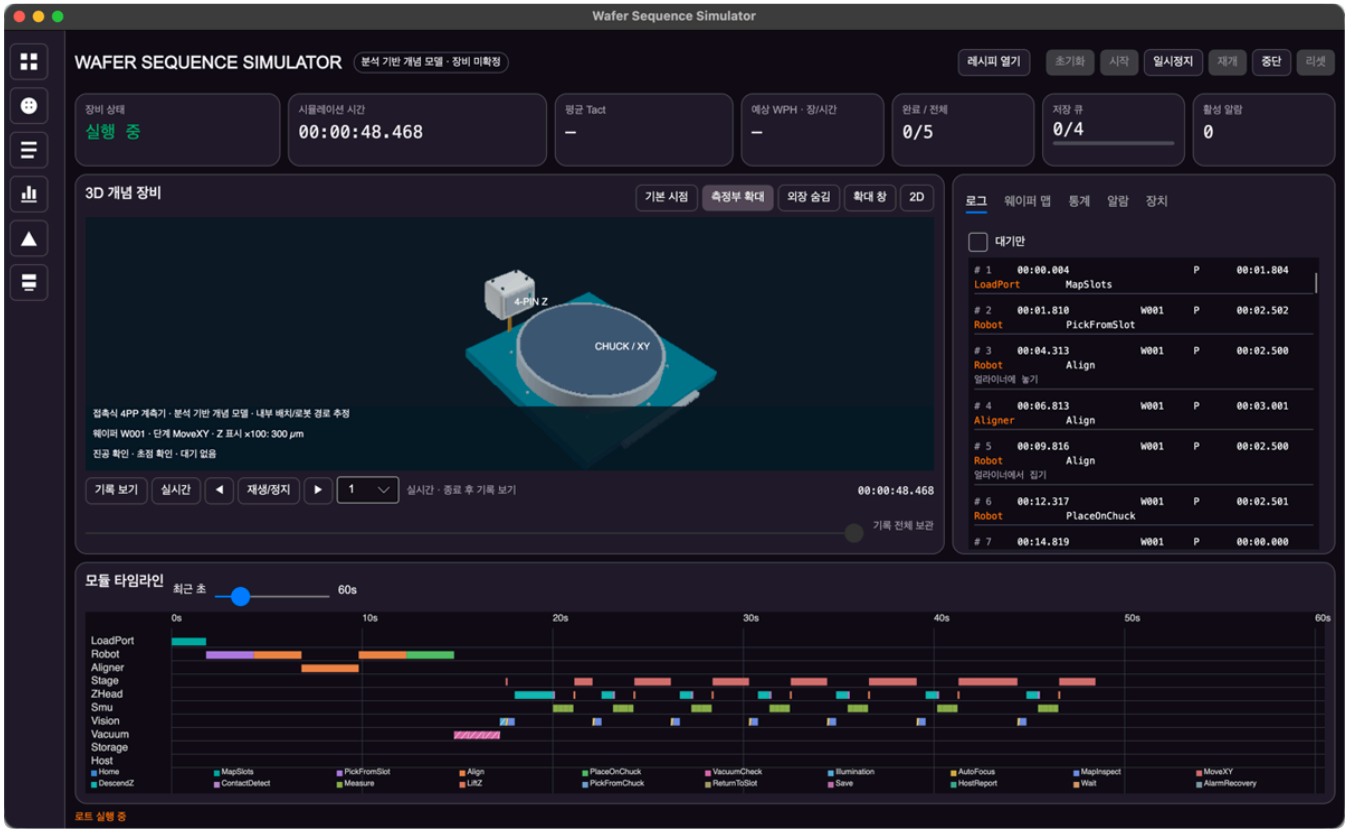
이 포트폴리오가 답하려는 질문은 하나다. "웨이퍼 한 장이 왜 이만큼 걸리고, 설정 하나를 바꾸면 어느 단계가 얼마나 줄어드는가." 그 답을 로그·간트·웨이퍼 맵·3D 재생으로 보여 준다.

2. 화면

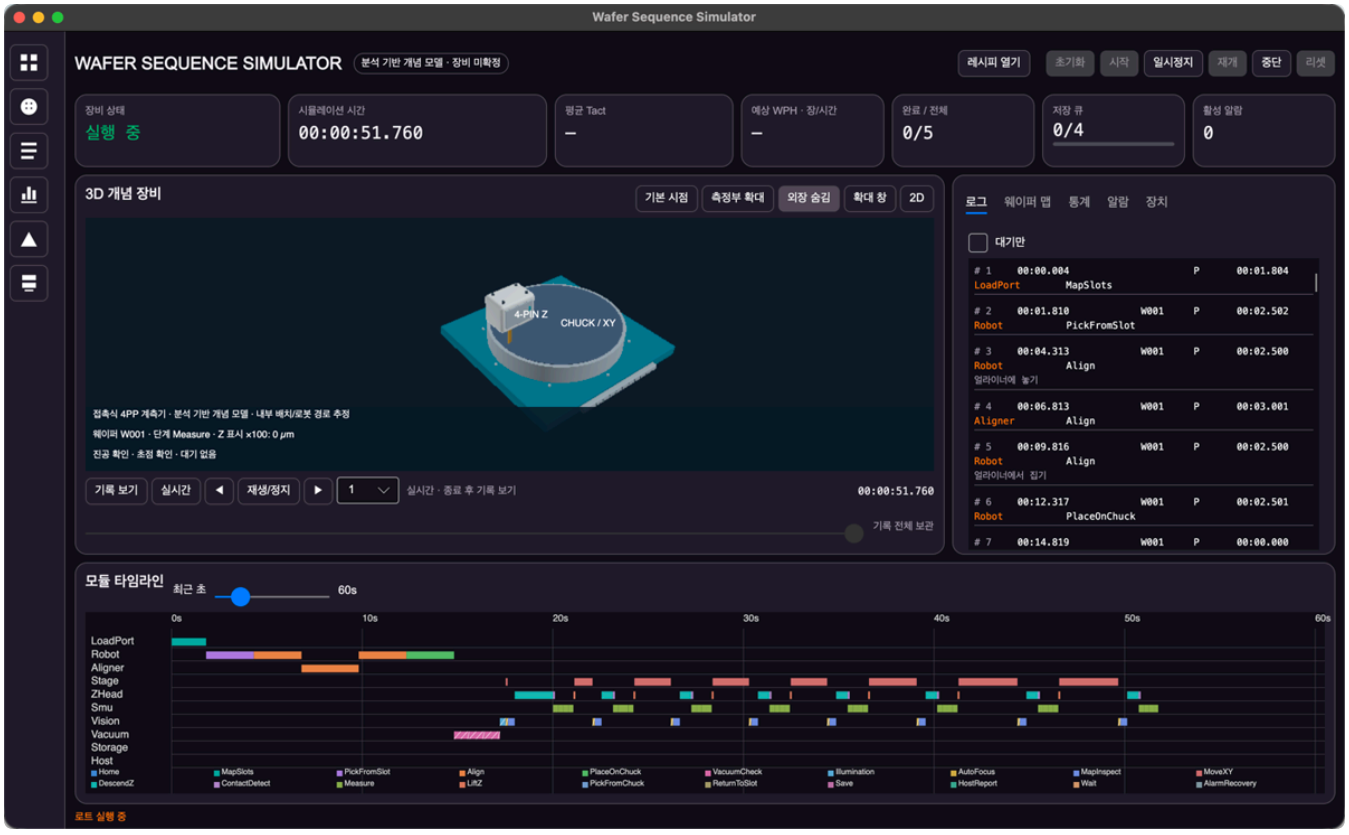
아래는 2026-10-13 오전 맥에서 Avalonia 화면을 직접 조작하며 찍은 실행 화면이다(시연용 가공 없음). 마지막 한 장은 GitHub Actions 윈도우 러너가 WinForms 앱을 창 없이 돌리며 저장한 것이다.



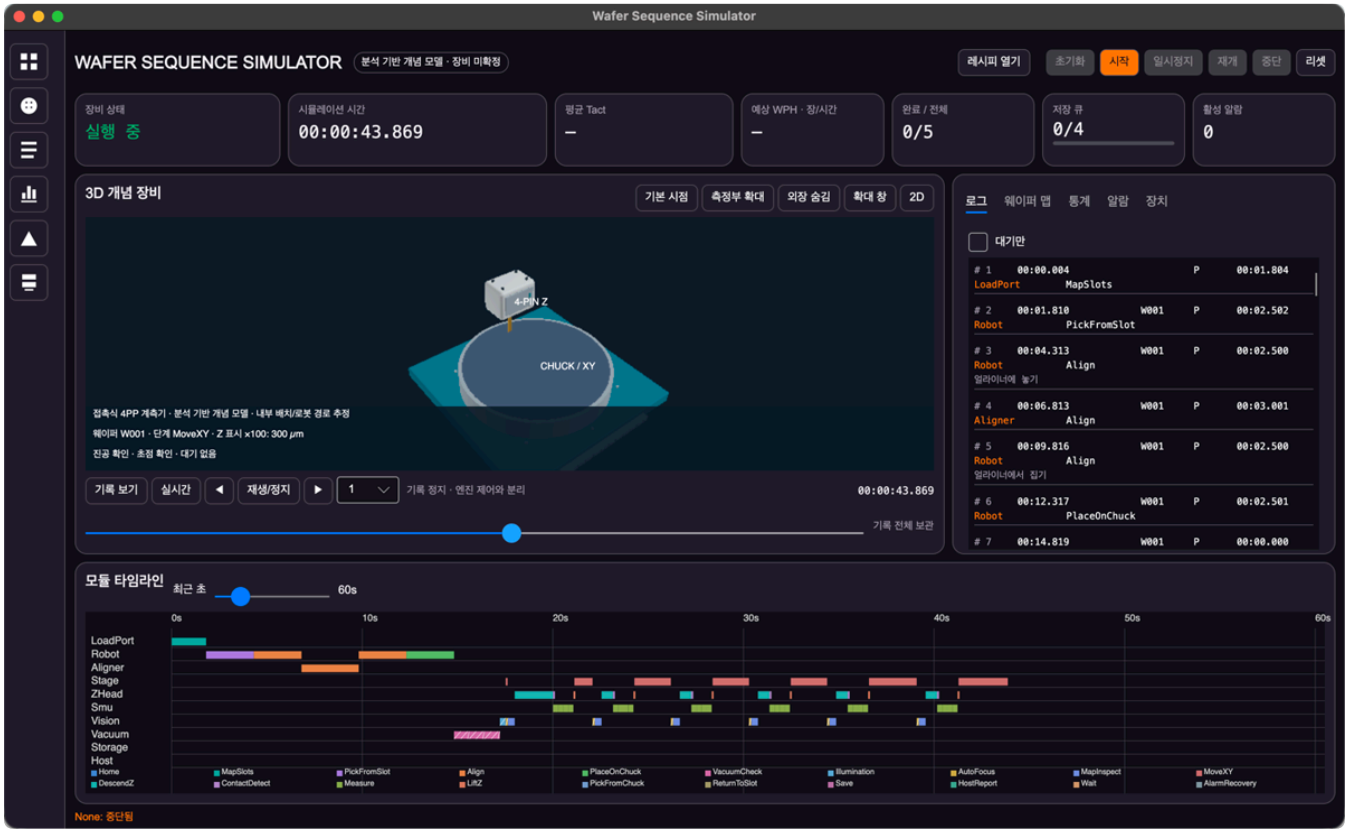
실행 중 — 3D 개념 장비 기본 시점 · 웨이퍼 W001이 척 위에서 Z 하강 중(시뮬레이션 45초). 왼쪽 위 지표, 가운데 3D, 오른쪽 로그(단계별 시각·소요·사유), 아래 모듈 타임라인(빗금 = 고정 대기).



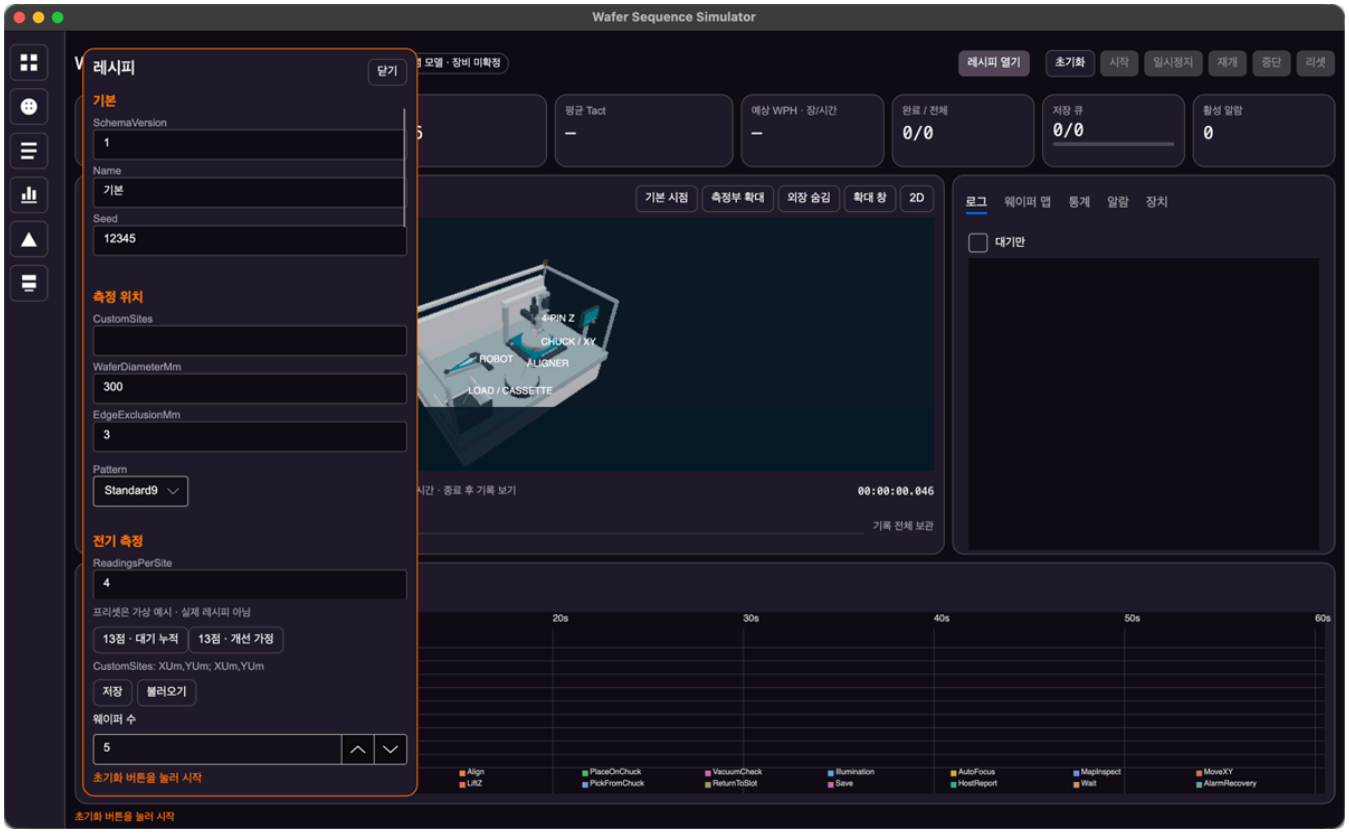
측정부 확대 · 4핀 Z 헤드가 척 위 사이트로 이동하는 순간. 「Z 표시 ×100」은 미세 Z 변위를 100배로 그린다는 표시다.



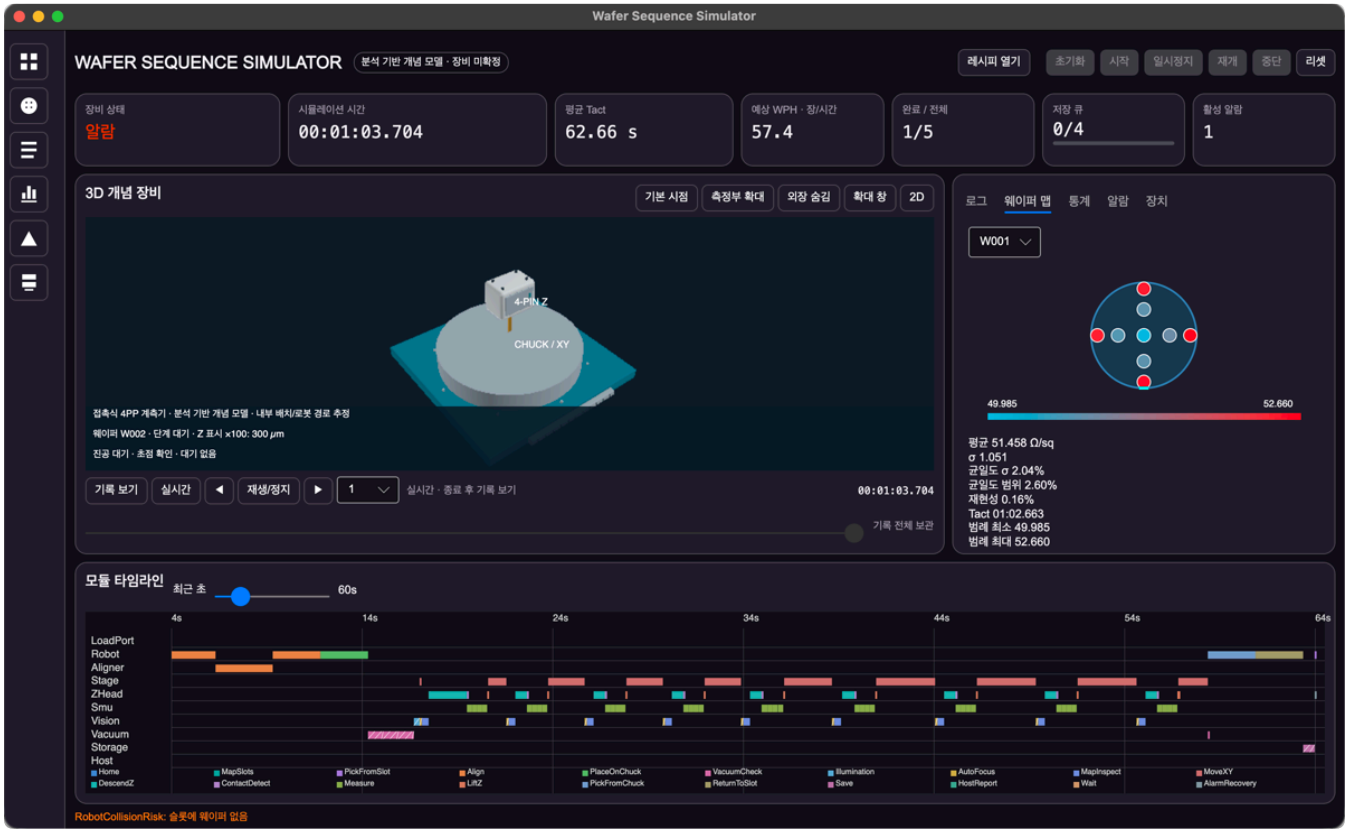
외장 숨김 · 캐비닛을 걷고 측정 중인 헤드와 척만 본다(단계 Measure, Z 0 μm = 접촉).

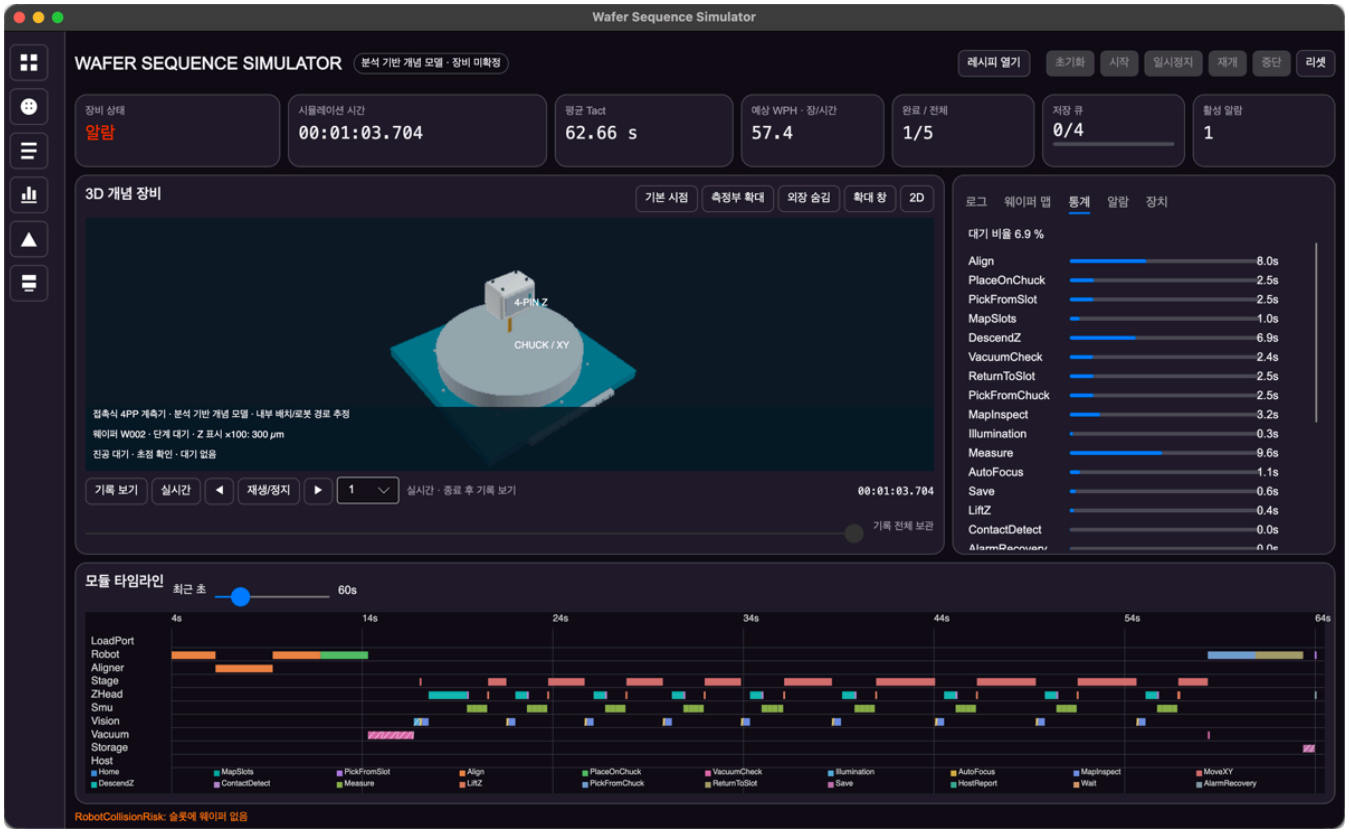


기록 재생 · 중단 후 「기록 보기」로 들어가 타임슬라이더를 43.9초로 되돌린 상태. 3D·로그·타임라인이 같은 시각을 따르고, 재생은 엔진에 명령을 보내지 않는다("기록 정지 · 엔진 제어와 분리").



레시피 패널 · 그룹별 입력, 가상 실험 프리셋 2종(13점 대기 누적 / 개선 가정), JSON 저장·불러오기.





웨이퍼 맵과 통계 · 첫 웨이퍼 완료 후(Tact 62.66초). 사이트별 Rs 컴러맵·평균·σ·균일도·재현성(위), 단계별 누적 시간과 대기 비율(아래). 이 캡처에 보이는 알람은 중단된 이전 로트가 척에 남긴 웨이퍼 때문에 다음 로트가 빈 슬롯을 집으려 한 것으로, 이 문서 작성 중 발견해 매핑된 점유 슬롯에서 웨이퍼를 배정하도록 고쳤다.



WinForms, CI 윈도우 러너 캡처 · 진짜 엔진으로 로트를 돌리다 첫 웨이퍼 네 번째 사이트 측정 순간에 `DrawToBitmap`으로 저장. 사람 손을 거치지 않은 캡처다.

3. 무엇을 시뮬레이션하나

웨이퍼 1장은 「픽업 → 정렬 → 척 탑재 → 진공 확인 → 조명 → [사이트마다: XY 이동 → (오토포커스·맵 검사) → Z 하강 → 접촉 감지 → 전압 N회 측정 → 리프트] → 안전 높이 → 반출 ‖ 저장」 순서로 흐른다. 로드포트·로봇·얼라이너·스테이지·Z 헤드·SMU·비전·진공·저장·호스트 열 모듈이 각자 상태기계를 갖고 동시에 움직인다.

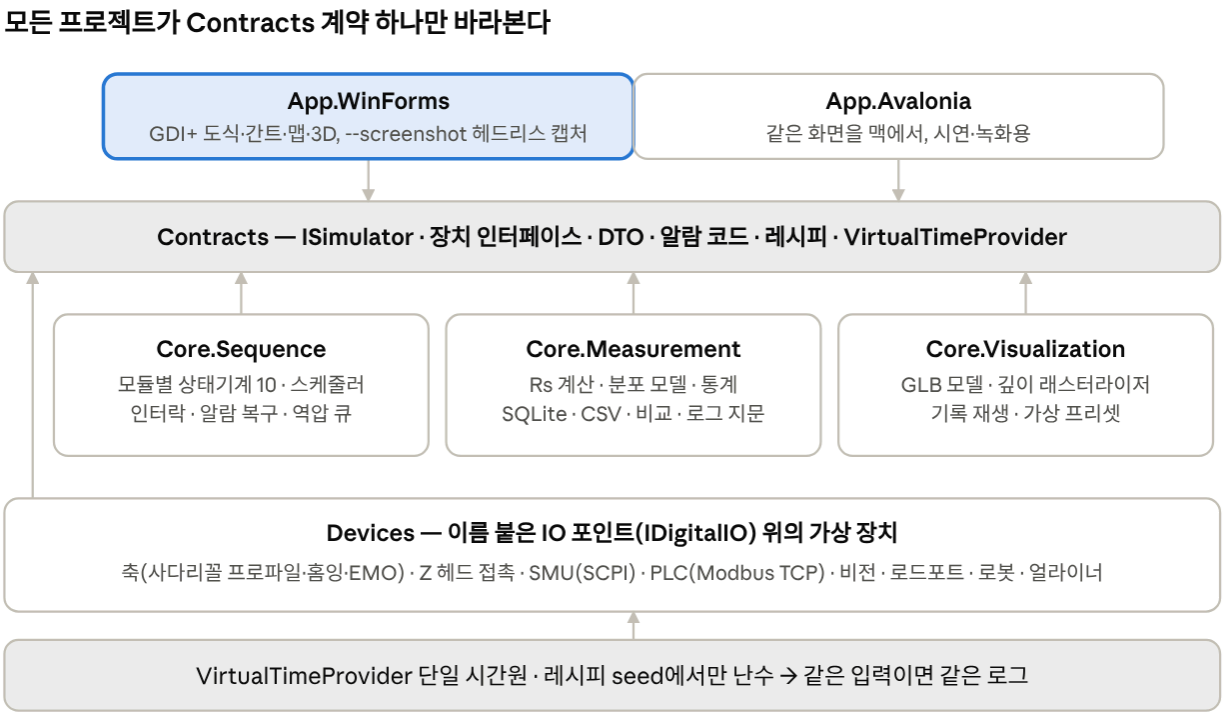
레시피로 바꿀 수 있는 것

묶음	항목	무엇을 보여주나
측정 위치	1/5/9/49점 표준 패턴·사용자 지정, 사이트당 반복 측정 수·1회 측정 시간	사이트 수가 Tact를 어떻게 쌍곡선으로 깎는지
모션	이동 속도·가속(사다리꼴 프로파일), Z 접근 방식(연속 vs 스텝), 사이트 사이 리프트 높이, 접촉 감지 타임아웃·재시도	물리 운전 조건이 사이트당 시간에 미치는 영향
고정 대기	진공 확인 횟수·시간, 조명 안정, 오토포커스, 맵 검사 범위(전수 vs 첫 사이트만)	장비 설정에 흔히 숨어 있는 상수 대기가 간트에 빗금으로 드러남
핸들링 중첩	측정 중 다음 웨이퍼를 미리 정렬, 반출과 저장을 겹침	자원 점유 스케줄러가 총 시간을 줄이는 정도
결과 저장	동기(시퀀스가 기다림) vs 비동기 큐(용량·쓰기 시간)	큐가 차면 측정값을 버리는 대신 시퀀스가 기다리는 역압
시계	실시간(배속) vs 즉시 계산	같은 레시피+seed면 어느 모드에서도 같은 로그

측정 데이터 · 가상 SMU가 SCPI 명령(`:SOUR:CURR`, `:MEAS:VOLT?`)으로 돌려주는 전압을 Rs = 4.532·V/I(가장자리 보정)로 계산한다. 분포 모델은 웨이퍼별 seed로 정해지는 방사형 분포 + 잡음이다. 웨이퍼마다 평균·σ·균일도 2종·재현성(같은 사이트 반복 σ)을 내고 컴러맵으로 그린다. 실행 이력은 SQLite에 쌓여 두 실행을 단계별로 비교할 수 있다.

가상 실험 프리셋 2종 · 「13점 · 대기 누적」(진공 확인 3회, 사이트 사이 Z 상승 600 µm, 맵 전수 검사, 동기 저장)과 「13점 · 개선 가정」(고정 대기 단축, Z 상승 300 µm, 맵 첫 사이트만, 비동기 저장)을 같은 seed로 돌려 단계별 차이를 비교한다. 둘 다 공개 기본값에서 만든 가상 예시이며 어떤 실제 장비의 분석을 재현한 것이 아니다.

4. 구조



화살표 = 의존 방향. 장치와 화면은 서로를 모르고, 계약을 통해서만 만난다.

의존 구조 · 프로젝트 8개

화면과 장치는 서로를 모른다. 둘 다 Contracts만 참조하고, 조립 루트 한 곳(각 앱의 `Services`)에서만 진짜 엔진·장치·저장소를 묶는다. 그래서 화면을 가짜 시뮬레이터로 먼저 만들고 나중에 엔진을 끼울 수 있었다.

프로젝트	역할
`src/Contracts`	계약만. `ISimulator`, 장치 인터페이스, DTO, 알람 코드 21종, 레시피, `VirtualTimeProvider`, UI 개발용 `FakeSimulator`
`src/Core.Sequence`	시퀀스 엔진. 모듈별 상태기계 10개, 웨이퍼 추적, 자원 점유 스케줄러(파이프라인), `Step` 인터락 게이트, EMO 감시, 알람·안전 상태·복구, 저장 큐(역압), 단계 로그·통계
`src/Core.Measurement`	Rs 계산·가장자리 보정, 면저항 분포 모델, 측정 패턴, 통계·재현성, SQLite 이력, 실행 비교, 레시피 JSON, CSV, 로그 지문
`src/Core.Visualization`	3D 개념 장비: GLB 부품을 엔진 상태에 연결, 픽셀 깊이 버퍼 래스터라이저(법선·금속성·근접면 클리핑), 실행 기록(`RunReplay`, 스냅샷 30,000·이벤트 5,000), 가상 실험 프리셋

프로젝트	역할
`src/Devices`	이름 붙은 IO 포인트 위의 장치: 축(사다리꼴 프로파일·소프트 리밋·홈잉·EMO 정지), Z 헤드 접촉 커플링, SCPI 가상 SMU, Modbus TCP 가상 PLC, 비전·로드포트(25슬롯)·로봇·얼라이너, `WaitFor(조건, 타임아웃, 알람코드)`
`src/App.WinForms`	C# WinForms HMI. GDI+ 도식·간트·맵·3D 패널. `--screenshot` 헤드리스 캡처(CI)
`src/App.Avalonia`	같은 화면을 Avalonia로. 맥 실행·시연용
`tests/Tests`	xUnit 89건

5. 설계 결정 5가지

면접에서 "이 줄 왜 이렇게 했나"를 물으면 답할 수 있어야 하는 결정들이다.

1. **모듈별 상태기계와 스케줄러.** 장비 상태를 열거형 하나로 두면 "언로딩 중첩" 같은 동시 동작을 표현할 수 없다. 로드포트·로봇·얼라이너·스테이지·Z 헤드·SMU·비전·진공·저장·호스트가 각자 상태를 갖고, 척·로봇·얼라이너는 자원으로 점유·해제한다. 장비 전체 상태는 모듈 상태에서 계산한다.
2. **시간원은 하나, 난수는 seed에서만.** `DateTime.Now` · `Stopwatch` · `Task.Delay(ms)` 를 금지하고 `VirtualTimeProvider` 하나로만 기다린다. 즉시 모드는 가상 타이머 큐를 가장 이른 순서로 발화시켜 겹친 동작은 겹친 만큼만 시간이 흐른다. 같은 레시피+seed면 로그가 같고, 로그 지문(SHA-256)으로 재현을 검증한다. 통신 타임아웃만은 현실의 성질이라 벽시계로 재다.
3. **측정값은 절대 버리지 않는다.** 비동기 저장 큐가 가득 차면 시퀀스가 「저장 큐 가득」 대기로 멈추고 그 대기가 간트에 보인다. 알람·중단·저장 실패 어느 경로로 끝나도 큐를 비우고 실패한 실행도 이력에 남긴다.
4. **인터락은 시퀀스 if문이 아니라 명령 계층에.** XY 이동·Z 하강·로봇 진입·진공 해제 직전에 `Step` 게이트가 사유와 함께 거부하고, 일시정지 뒤 재개 시 재검사한다. EMO는 읽기 실패도 차단(fail-closed)이고, 알람이 나면 SMU 출력 차단→축 정지→Z 상승의 안전 상태로 간 뒤 운영자 Ack→Reset→재초기화 경로를 탄다.
5. **IO 포인트 추상화 위에 장치와 프로토콜.** 가상 장치도 실제처럼 "명령 → 상태 폴링 → 타임아웃 → 알람 코드"로 동작한다. 같은 장치 묶음을 인메모리 전송과 SCPI/Modbus TCP 전송으로 바꿔 끼울 수 있고, 통합 테스트는 두 경로를 모두 돈다. 레시피 seed와 웨이퍼 ID는 엔진이 `IDeviceContext` 로 장치에 직접 넘긴다.

여기에 3D 고도화에서 더한 원칙 둘: **회전·확대는 관찰 도구일 뿐 장비의 시간을 바꾸지 않고**, 3D·로그·타임라인의 재생 시각은 항상 일치한다. **기록 재생은 엔진에 명령을 보내지 않는다** — 재생 중 조작 버튼은 실제 엔진 상태를 따른다.

6. 공고 담당업무와의 대응

장비 소프트웨어 엔지니어 공고의 담당업무를 기준으로, 이 저장소에서 그 근거가 어느 클래스인지 적었다. 없는 것은 없다고 썼다.

공고 항목	저장소의 근거	정도
C# WinForms/.NET 장비 제어 앱	`App.WinForms/MainForm.cs` (SplitContainer·ToolStrip·ListView VirtualMode·TabControl), `Panels.cs` (GDI+ 도식·간트·맵), `Equipment3DPanel.cs`, `--screenshot` 헤드리스 캡처	구현
모션 제어·DIO·인터락·시퀀스 설계	`Core.Sequence/SequenceEngine` (모듈별 상태기계·스케줄러· `Interlock` ·EMO 감시), `Devices/SimulatedAxis` (사다리꼴 프로파일·소프트 리밋·홈잉·EMO 정지), `IDigitalIO` 포인트	가상 구현
PLC·Modbus TCP·TCP/IP·Serial 통신	`Devices/Modbus.cs` (FC1/3/5/6/16 클라이언트·가상 PLC, 인메모리·TCP 전송, 취소 후 재동기화), `Devices/Scpi.cs` (SMU)	Modbus TCP·SCPI 구현, EtherNet/IP·Serial 미구현
상태 모니터링·알람·로그·이력	`EquipmentSnapshot` (30 Hz), `Alarm` / `AlarmCode` 21종·Ack·Reset, `StepEvent` 로그, SQLite 이력, `RunReplay` 기록 재생	구현
사내 공용 제어 플랫폼	`Contracts` (계약만) + `DeviceSetFactory` (같은 장치 묶음을 인메모리·프로토콜·TCP로 교체)	구조로 답함
측정 데이터 처리·통계·차트·맵 UI	`Core.Measurement/RsCalculator` · `ResistanceField`, 웨이퍼 맵·통계 막대·간트(두 UI)	구현
리포트·DB 출력·상위 연동	`CsvExporter`, `SqliteResultStore`, `RunComparer`, SECS/GEM은 CEID/ALID 매핑 설계 문서만	부분
정확도·재현성 데이터 분석	사이트 반복 σ, 균일도 2종, 로그 지문(SHA-256)으로 재현 검증, seed 고정	구현
하드웨어·기계·전장 협업	`IoPoints` (전장 IO 맵 이름), 3D 개념 모델(부품 역할을 엔진 상태에 연결), 개발 이전 CAD·시제품 경력	간접
현장 셋업·이슈 대응	레시피 JSON 저장·불러오기, 기록 재생으로 "그때 무슨 일이 있었나" 되감기	부분
요구사항~문서화	README, 공고 매핑, SECS 매핑, 3D 참고 근거 문서, 커밋 15개의 순서	구현

자격·우대 항목 중 직접 답한 것: 멀티스레드·async(전 계약이 async+CancellationToken, UI 디스패처 규약), Git, 로그·재현으로 이슈 추적(재현성 설계 자체), SQLite, 소규모 팀 모듈 완결, Claude AI 활용. 답하지 못한 것: 실장비 현장 경험, .NET Framework 마이그레이션(설계 메모만), MSSQL/PostgreSQL.

7. 검증

무엇을	어떻게	결과 (2026-10-13 기준)
단위·통합 테스트	`dotnet test` (xUnit)	89건 통과. 시퀀스(결정성·역압·중첩·알람 복구·인터락·EMO·Pause/Abort·RunId), 측정, 장치(프로파일·커플링·SCPI·Modbus TCP·seed), 조립 통합(인메모리·프로토콜·실시간·13점 프리셋·기록 재생·깊이 래스터·정밀 모델)
재현성	같은 레시피+seed 두 번 실행, 로그 지문 비교	동일. 전체 테스트 5회 반복에서도 흐들림 없음
CI	GitHub Actions: ubuntu 빌드·테스트 + windows 러너 WinForms 빌드·실행·화면 4장 저장	커밋 d654708(1차 완성)까지 두 잡 성공. 3D 고도화 커밋(5933578)은 push 직후 실행 중
실제 실행	맥에서 Avalonia 화면을 직접 조작	5장 로트 완주(평균 Tact 약 63초, UPH 약 57, 1차 완성 기준). 3D 고도화판은 실행·시점 전환·중단·기록 재생까지 조작해 위 화면으로 남김
독립 리뷰	모듈별 리뷰 5회 + 커밋 전 코드 리뷰	발견(리스트 공유로 요약이 지워지는 버그, 같은 RunId 덮어쓰기, 동기 Modbus 읽기가 스냅샷을 막음, EMO 읽기 실패를 해제로 해석 등)을 반영
기밀 검사	특정 회사·장비·수치 용어 grep	없음. 모든 수치·배치는 가상

맥에서는 WinForms를 실행할 수 없어 컴파일만 했고, 실행과 화면은 CI 윈도우 러너의 캐처로만 확인했다.

8. 한계와 제작 과정

한계

- 실제 모션 보드·PLC·SMU·SECS/GEM은 연동하지 않았다. 장치는 전부 가상이고, 프로토콜은 인메모리·TCP 가상 서버와 주고받은 것만 확인했다. EtherNet/IP·Serial은 전송 인터페이스 뒤에 붙일 자리만 있다.

- WinForms는 맥에서 컴파일만 되고 실행은 윈도우(CI)에서만 확인했다. 사람이 창을 직접 조작한 것은 Avalonia 쪽이다.
- 3D 모델은 공개된 제조사 외형 사진을 참고한 개념 모델이다. 내부 배치·크기·로봇 관절 궤적은 설계 가정이며 어떤 실장비의 도면도 아니다. 래스터라이저는 CPU 기반이라 큰 창에서는 프레임이 늘어진다(렌더 면적 81만 픽셀 상한).
- MSSQL·PostgreSQL은 `IResultStore` 뒤에 붙이는 자리만 있다(기본 SQLite). .NET Framework 앱에서 쓰려면 `Contracts` · `Core` 를 netstandard2.0으로 멀티타깃하면 된다(미구현).
- 즉시 모드의 가상 타이머 스케줄러는 "스레드풀이 조용해지면 다음 타이머"라는 휴리스틱을 쓴다. CPU를 많이 쓰는 콜백이 섞이면 순서가 흔들릴 수 있어 테스트를 반복 실행으로 확인했다.

제작 과정

2026년 10월 12일 저녁, 3시간 안에 1차본을 끝내는 것을 목표로 했다. 계약(`src/Contracts`) 설계와 모듈 경계, 작업 분배, 통합, 검증은 내가 했고, 구현은 Claude Code로 조율한 작업자 에이전트(Codex) 5명이 모듈별로 병렬로 썼다. 모듈마다 독립 리뷰 에이전트가 코드를 읽어 발견한 것(요약이 호출자 리스트를 공유해 다음 웨이퍼에 지워지는 버그, 같은 RunId 덮어쓰기, 동기 Modbus 읽기가 스냅샷을 막는 문제, EMO 읽기 실패를 해제로 해석하는 문제 등)을 반영했다. 통합에서 직접 고친 것은 정적 초기화 데드락, 즉시 모드 스케줄러, 척 재석 센서 커플링, 대기 헬퍼의 경과 측정, 엔진이 UI 스레드를 붙잡던 정지, 로드포트 슬롯 점유 기본값이다. 이틀째는 3D 개념 장비와 기록 재생을 더했다. 커밋 15개에 그 순서가 남아 있다.

C# WinForms는 이 프로젝트로 처음 썼다. 언어를 며칠 만에 익혀 돌아가는 것까지 포트폴리오에 담은 이유는, 언어 공백을 숨기는 대신 학습 속도로 보여 주는 쪽이 정직하다고 생각했기 때문이다.